// BLACK PEN Answer all questions first. // GREEN PEN Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. A simplifying rule in a model.

Q2. Invalid values can violate preconditions.

Q3. A method/module designed for use in several parts of software.

Q4. C# array [2,3,5,7]: set a[1]=a[0]+a[2]; give new array.

You Do pages · Independent questions

Q1. Inputs count and unit price [2], numeric total [1], count non-negative [1], price non-negative [1].

Q2. Reused expensive lookup [2], cache read avoids recomputation [1], stale cached value [1], time-to-live or version invalidation [2].

Q3. Method accepts room ID [1], caches keyed room data [2], refresh on change/expiry [1], tests repeat hit and outdated room change [2].

Exit Ticket · Model answer

False. A cached result can become stale.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 061 · DRAFT

// BLACK PEN Answer all questions first. // GREEN PEN Then: tick and rewrite mistakes.

LESSON 061 · Lesson 061 · Thinking Ahead: Inputs, Preconditions and Reuse

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is an abstraction assumption?

Q2. Why validate inputs before processing?

Q3. What is a reusable component?

Q4. 017

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

precondition — Requirement for valid use.

postcondition — Expected condition after execution.

cache — Saved result for quicker reuse.

module — Reusable program component.

New learning

Identify — Specify the inputs and outputs before choosing algorithms: ticket count and unit price determine total charge.

Model — Make preconditions explicit. Invalid negatives or overflow can produce incorrect outputs; validation and appropriate types reduce risks.

Evaluate — Caching saves a previously computed result for reuse, trading memory and stale-result risk against repeated calculation time. Reusable methods reduce repeated code.

Worked example

A function doubles an integer. State input/output and a precondition if it accepts values 0–50 only.

1. Explain the decision — The algorithm's result can be correct mathematically but invalid under the declared application rules.

We Do · Practise with guidance

A weather app caches a temperature for 10 minutes. Give a benefit and drawback.

```
Avoids frequent costly API calls; can display a stale temperature until cache
  refresh.
```

Explain the decision

Explain caching's time/network benefit and correctness trade-off.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

For function TicketCost(count,price), state two inputs, output and one precondition. [5]

Question 2

Give a scenario where caching a result saves time, plus a stale-data risk and one invalidation strategy. [6]

Question 3

An API-dependent school timetable app calls same room lookup 200 times. Design a reusable method with cache and describe two tests. [6]

Exit Ticket · Answer before checking

Caching a result can never affect correctness.